

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
имени М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

ОТЧЕТ ПО ЗАДАНИЮ №6

**«Сборка многомодульных программ.
Вычисление корней уравнений и определенных
интегралов»**

Вариант 9

Исполнитель: студентка 104 группы
Ларичева Д. А.

Преподаватель:
Гуляев Д. А.

Москва, 2026

Содержание

Постановка задачи	2
Математическое обоснование	3
Графики функций	4
Результаты экспериментов	5
Количество итераций	5
Результаты тестирования	6
Структура программы и спецификация функций	7
main.c	7
numerical.c	7
functions.asm	8
Сборка программы (Make-файл)	9
Отладка программы, тестирование функций	10
Программа на Си и на Ассемблере	11
Анализ допущенных ошибок	12
Литература	13

Постановка задачи

Целью работы является разработка многомодульной программы на языках C и NASM32 для вычисления площади плоской фигуры, ограниченной тремя кривыми.

В варианте 9 заданы функции:

$$f_1(x) = \frac{3}{(x-1)^2 + 1},$$

$$f_2(x) = \sqrt{x + 0.5},$$

$$f_3(x) = e^{-x}.$$

По индивидуальному варианту необходимо использовать:

- точности вычислений: $\varepsilon = 10^{-3}, 10^{-5}, 10^{-7}$;
- тип вычислений: `float`;
- методы поиска корней: метод деления пополам и метод Ньютона;
- метод вычисления интегралов: метод прямоугольников.

Требуется найти точки пересечения кривых, представить площадь фигуры как сумму определенных интегралов и вычислить ее с заданной точностью.

Функции f_1, f_2, f_3 и их производные реализуются на языке NASM32. Остальная часть программы реализуется на языке C. Сборка программы выполняется с помощью `Makefile`.

Математическое обоснование

Точки пересечения двух кривых находятся как корни уравнения:

$$f_i(x) = f_j(x).$$

Для применения численных методов это уравнение приводится к виду:

$$F(x) = f_i(x) - f_j(x) = 0.$$

В работе используются три уравнения:

$$f_1(x) - f_3(x) = 0,$$

$$f_2(x) - f_3(x) = 0,$$

$$f_1(x) - f_2(x) = 0.$$

Для поиска корней были выбраны следующие отрезки:

- $f_1 = f_3: [-0.5; 0];$
- $f_2 = f_3: [0; 0.5];$
- $f_1 = f_2: [1.5; 2.5].$

Выбор отрезков был сделан по графикам функций и проверке смены знака функции $F(x)$ на концах отрезка.

Метод деления пополам основан на следующем свойстве: если функция непрерывна на отрезке $[a, b]$ и выполняется условие

$$F(a)F(b) < 0,$$

то на этом отрезке существует корень. На каждом шаге вычисляется середина отрезка:

$$c = \frac{a + b}{2}.$$

Затем выбирается та половина отрезка, на которой функция меняет знак.

Метод Ньютона задается формулой:

$$x_{n+1} = x_n - \frac{F(x_n)}{F'(x_n)}.$$

Для этого нужны производные функций:

$$f_1'(x) = \frac{-6(x-1)}{((x-1)^2 + 1)^2},$$

$$f_2'(x) = \frac{1}{2\sqrt{x+0.5}},$$

$$f_3'(x) = -e^{-x}.$$

Метод Ньютона при хорошем начальном приближении сходится быстрее метода деления пополам [1].

Для вычисления интегралов используется метод прямоугольников:

$$\int_a^b f(x) dx \approx h \sum_{k=0}^{n-1} f(x_k),$$

где

$$h = \frac{b-a}{n}.$$

Количество разбиений увеличивается до тех пор, пока разность между двумя последовательными приближениями не станет меньше заданной точности.

Графики функций

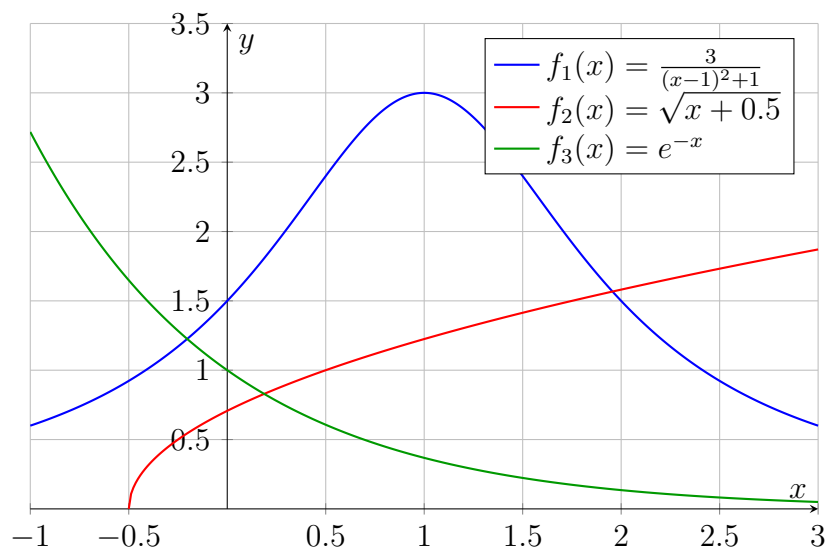


Рис. 1: Графики функций варианта 9

Результаты экспериментов

В результате работы программы были получены точки пересечения, представленные в таблице.

Кривые	x	y
$f_1 = f_3$	-0.2033349	1.2254827
$f_2 = f_3$	0.1874113	0.8291027
$f_1 = f_2$	1.9561526	1.5672110

Таблица 1: Координаты точек пересечения

Площадь фигуры была вычислена как сумма интегралов на промежутках между найденными точками пересечения.

На промежутке от x_{13} до x_{23} область ограничена функциями f_1 и f_3 .

На промежутке от x_{23} до x_{12} область ограничена функциями f_1 и f_2 .

Итоговое значение площади:

$$S \approx 2.3386023.$$

Количество итераций

Для метода деления пополам были получены следующие значения:

Уравнение	Количество итераций
$f_1 = f_3$	15
$f_2 = f_3$	15
$f_1 = f_2$	16

Таблица 2: Количество итераций метода деления пополам

Для метода Ньютона были получены следующие значения:

Уравнение	Количество итераций
$f_1 = f_3$	3
$f_2 = f_3$	3
$f_1 = f_2$	3

Таблица 3: Количество итераций метода Ньютона

Из таблиц видно, что метод Ньютона для выбранных отрезков сошелся быстрее метода деления пополам.

Результаты тестирования

Тест	Получено	Ожидалось
Бисекция $f_1 = f_3$	-0.2033310	-0.2033349
Ньютон $f_1 = f_3$	-0.2033349	-0.2033349
Бисекция $f_2 = f_3$	0.1874161	0.1874113
Ньютон $f_2 = f_3$	0.1874113	0.1874113
Бисекция $f_1 = f_2$	1.9561539	1.9561525

Таблица 4: Тестирование поиска корней

Интеграл	Получено	Ожидалось
$\int_0^1 f_1(x) dx$	2.3561982	2.3561945
$\int_0^2 f_2(x) dx$	2.3995331	2.3995291
$\int_0^1 f_3(x) dx$	0.6321140	0.6321206

Таблица 5: Тестирование вычисления интегралов

Структура программы и спецификация функций

Программа состоит из следующих файлов:

```
lab6/  
  asm/  
    functions.asm  
  include/  
    functions.h  
    numerical.h  
  src/  
    main.c  
    numerical.c  
  Makefile
```

main.c

Файл `main.c` является главным модулем программы. В нем выполняется обработка аргументов командной строки, вызов численных методов и вывод результатов.

Поддерживаются следующие режимы:

- вывод площади фигуры;
- вывод точек пересечения;
- вывод количества итераций;
- тестирование поиска корней;
- тестирование интегралов.

numerical.c

Файл `numerical.c` содержит реализацию численных методов:

- `root_bisection` — метод деления пополам;
- `root_newton` — метод Ньютона;
- `integral_rect` — метод прямоугольников.

functions.asm

Файл `functions.asm` содержит реализацию функций и их производных на языке NASM32:

- `f1;`
- `f2;`
- `f3;`
- `df1;`
- `df2;`
- `df3.`

Функции вызываются из С-кода по соглашению `cdecl`.

Сборка программы (Make-файл)

Сборка программы выполняется с помощью Makefile. Используются компилятор gcc, ассемблер nasm и ключ -m32 для сборки 32-битной программы.

Основные команды:

```
make
make METHOD=NEWTON
make EPS=1e-3
make EPS=1e-5
make EPS=1e-7
make clean
```

Команда make собирает программу с методом деления пополам.

Команда make METHOD=NEWTON собирает программу с методом Ньютона.

Команда make EPS=... позволяет задать точность на этапе сборки.

Команда make clean удаляет объектные файлы и исполняемый файл.

Пример команд компиляции:

```
gcc -m32 -Wall -Wextra -O2 -Iinclude
nasm -f elf32
gcc -m32 -o prog ...
```

Отладка программы, тестирование функций

Для проверки корректности были реализованы специальные режимы запуска.

Тестирование поиска корней:

```
./prog --test-root
```

Пример вывода:

```
bisection f1=f3: x=-0.2033310, expected about -0.2033349
newton f1=f3: x=-0.2033349, expected about -0.2033349
bisection f2=f3: x=0.1874161, expected about 0.1874113
newton f2=f3: x=0.1874113, expected about 0.1874113
bisection f1=f2: x=1.9561539, expected about 1.9561525
```

Тестирование интегралов:

```
./prog --test-integral
```

Пример вывода:

```
integral f1[0,1] = 2.3561982, expected about 2.3561945
integral f2[0,2] = 2.3995331, expected about 2.3995291
integral f3[0,1] = 0.6321140, expected about 0.6321206
```

Также были проверены параметры:

- --help;
- --roots;
- --iterations;
- --test-root;
- --test-integral.

Программа на Си и на Ассемблере

Программа реализована на языках С и NASM32.

На языке ассемблера реализованы функции варианта и их производные. На языке С реализованы численные методы и основная логика программы.

Исходные тексты программы находятся в архиве, приложенном к отчету.

В архив входят:

- `main.c`;
- `numerical.c`;
- `functions.asm`;
- `functions.h`;
- `numerical.h`;
- `Makefile`.

Анализ допущенных ошибок

Во время выполнения работы возникали следующие ошибки:

- сначала был выбран неправильный набор функций;
- были неправильно подобраны интервалы поиска корней;
- возникали ошибки при работе с FPU;
- возникали ошибки при подключении NASM-модуля к С-программе;
- метод Ньютона требовал аккуратного выбора начального приближения.

После исправления функций, интервалов и ассемблерной части программа начала корректно вычислять точки пересечения и площадь фигуры.

Литература

Список литературы

- [1] Ильин В. А., Садовничий В. А., Сендов Бл. Х. Математический анализ. Т. 1. — М.: Наука, 1985.
- [2] Трифонов Н. П., Пильщиков В. Н. Задания практикума на ЭВМ.